

#APLICAÇÃO QUE VISA A CONSTRUÇÃO DE UMA EMPRESA FICTÍCIA E SEU FUNCIONAMENTO COM UM BOT INTELIGENTE PARA DAR ASSISTENCIA AOS DEPATAMENTOS,

SIGA TODAS AS ORIENTAÇÕES

1. Empresa Fictícia: **EletroMax Distribuidora**

- **Sector:** Materiais elétricos, automação e iluminação.
- **Porte:** Médio, com 3 filiais (SP, RJ, MG) e ~200 clientes ativos.
- **Dores iguais à Contattos+:**
 - Estoque invisível (não sabem o que entrou vs. o que saiu).
 - Quebra de estoque (compram 100, aparecem 60 no sistema).
 - Margem apertada por falta de visibilidade de custo por fornecedor.
 - Decisões travadas porque relatórios demoram dias.
- **Nomes reais de fornecedores** (para dar realismo): Tramontina, Siemens, Weg, Schneider, Legrand, Pial, etc.

2. Modelo de Dados (MySQL)

```
``sql
-- Produtos (50 itens)
CREATE TABLE produtos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  codigo VARCHAR(20) UNIQUE,
  nome VARCHAR(100),
  categoria VARCHAR(50),
  unidade VARCHAR(10) -- ex: un, kg, m
);

-- Fornecedores (incluindo múltiplos CNPJs para o mesmo nome)
CREATE TABLE fornecedores (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome_fantasia VARCHAR(100),
  razao_social VARCHAR(100),
  cnpj VARCHAR(18) UNIQUE,
  grupo_id INT -- agrupa CNPJs do mesmo grupo econômico
);

-- Compras (notas fiscais de entrada)
CREATE TABLE compras (
  id INT PRIMARY KEY AUTO_INCREMENT,
  fornecedor_id INT,
  produto_id INT,
  data_emissao DATE,
  quantidade INT,
  valor_unitario DECIMAL(10,2),
  valor_total DECIMAL(10,2),
```

```
FOREIGN KEY (fornecedor_id) REFERENCES fornecedores(id),
FOREIGN KEY (produto_id) REFERENCES produtos(id)
);
```

```
-- Estoque atual (saldo)
CREATE TABLE estoque_atual (
  produto_id INT PRIMARY KEY,
  filial VARCHAR(2), -- SP, RJ, MG
  saldo_quantidade INT,
  ultima_atualizacao DATETIME,
  FOREIGN KEY (produto_id) REFERENCES produtos(id)
);
```

```
-- Histórico de movimentações (para simular 12 meses)
-- Podemos gerar a partir das compras e vendas (se houver), mas o escopo fala apenas entradas e
saldo.
-- Para a demo, podemos ter uma tabela de movimentos com entradas e saídas (se quisermos).
-- Mas o escopo cita "entradas e saldo", então podemos ter uma tabela de entradas e uma de estoque
atual.
-- Para calcular "quebra", comparamos o esperado (total de entradas - saídas) com o estoque real.
-- Se não tivermos saídas, podemos supor que o esperado é simplesmente o total de entradas
acumulado.
-- Vamos adotar: estoque esperado = total comprado - total vendido (se tivermos vendas).
-- Mas para simplificar, podemos ter uma tabela de saídas (vendas) também.
---
```

****Tabela adicional para simular divergência:****

```
```sql
-- Movimentos (entradas e saídas)
CREATE TABLE movimentos (
 id INT PRIMARY KEY AUTO_INCREMENT,
 produto_id INT,
 tipo ENUM('entrada', 'saida'),
 quantidade INT,
 data_movimento DATE,
 FOREIGN KEY (produto_id) REFERENCES produtos(id)
);

```

**\*\*Mas para a demo, podemos gerar dados sintéticos diretamente com scripts Node.js usando `faker` ou `chance`. Vou sugerir um seed.js que cria:\*\***

- 50 produtos com nomes reais (ex: "Cabo 10mm<sup>2</sup>", "Interruptor 3 polos", etc.).
- 5 fornecedores, cada um com 1 ou 2 CNPJs (grupo\_id).
- Compras (entradas) nos últimos 12 meses, com quantidades variadas.
- Estoque atual = total entrado - total saído (saídas geradas aleatoriamente) - uma divergência proposital em alguns produtos para gerar o alerta de quebra.

---

**## 3. Catálogo de Consultas (4 principais)**

Cada consulta é parametrizada e tem permissões associadas.

```
| **Consulta** | **Parâmetros** | **Perfil permitido** | **SQL exemplo** |
|-----|-----|-----|-----|
| **1. Estoque/giro por fornecedor** | fornecedor_id, período (mês/ano) | compras, financeiro |
`SELECT p.nome, SUM(c.quantidade) as total_entrado, e.saldo, (SUM(c.quantidade) - e.saldo) as
quebra FROM compras c JOIN produtos p ON c.produto_id=p.id JOIN estoque_atual e ON
p.id=e.produto_id WHERE c.fornecedor_id = ? AND c.data_emissao BETWEEN ? AND ? GROUP
BY p.id` |
| **2. Participação do fornecedor (agrupando CNPJs)** | fornecedor_grupo_id, período | compras |
`SELECT f.nome_fantasia, SUM(c.valor_total) as total FROM compras c JOIN fornecedores f ON
c.fornecedor_id=f.id WHERE f.grupo_id = ? AND c.data_emissao BETWEEN ? AND ? GROUP
BY f.id` |
| **3. Comparação entre períodos (ex: mês atual vs mês anterior)** | produto_id, mês1, mês2 |
compras, financeiro | `SELECT SUM(quantidade) FROM compras WHERE produto_id=? AND
MONTH(data_emissao)=? AND YEAR(data_emissao)=?` (duas vezes) |
| **4. Margem por produto/fornecedor** | fornecedor_id ou produto_id | **financeiro apenas** |
`SELECT AVG(valor_unitario) as custo_medio, (preco_venda - AVG(valor_unitario)) as margem
FROM compras c JOIN produtos p ON c.produto_id=p.id ...` (se tivermos preço de venda) |
```

**\*\*Permissões:\*\***

- Perfil "compras" pode ver 1, 2, 3.
- Perfil "financeiro" pode ver 1, 3, 4.
- Ambos podem ver estoque básico, mas margem só financeiro.

---

#### ## 4. Arquitetura (baseada no escopo)

...

[Web Chat (HTML/CSS/JS servido por Node)]

↑

[Orquestrador Node + Fastify]

↑

[Camada de Permissões (JSON fixo)]

↑

[Catálogo de Consultas (SQL + metadados)]

↑

[MySQL (dados sintéticos)]

↑

[IA (Haiku 4.5 via SDK ou Gemini)]

↑

[QuickChart.io (gráficos)]

...

**\*\*Fluxo:\*\***

1. Usuário digita pergunta no chat (com identificação simulada de perfil).
2. Orquestrador chama IA para extrair **\*\*intenção + parâmetros\*\*** em JSON (ex: ``{"intent":"estoque_fornecedor", "fornecedor":"Tramontina", "periodo":"2026-07"}``).
3. Valida permissão do perfil para aquela intenção.
4. Executa a consulta SQL parametrizada correspondente.

5. Formata resposta com IA (ou template) e gera gráfico (QuickChart) se solicitado.
6. Exibe no chat, incluindo **\*\*custo estimado da conversa\*\*** (tokens).
7. Se quebra de estoque > 10%, acende alerta amarelo no card.

---

## ## 5. Implementação Passo a Passo (1 semana)

### ### Dia 1 – Base de Dados + Seed

- Criar tabelas conforme modelo.
- Gerar 50 produtos (usar lista real).
- Inserir 5 fornecedores (Tramontina, Siemens, etc.) com 2 CNPJs cada.
- Gerar compras dos últimos 12 meses (média de 30 notas por mês).
- Gerar estoque atual com pequenas divergências (quebra) em 3-4 produtos.
- **\*\*Script:\*\*** `seed.js` com `mysql2` e `faker`.

### ### Dia 2 – Catálogo e Endpoints

- Criar arquivo `catalogo.js` com objetos definindo cada consulta:

```
``js
{
 id: 'estoque_fornecedor',
 sql: 'SELECT ... WHERE fornecedor_id = ? AND data BETWEEN ? AND ?',
 params: ['fornecedor', 'data_inicio', 'data_fim'],
 permissoes: ['compras', 'financeiro'],
 description: 'Estoque e quebra por fornecedor'
}
...

```

- Implementar endpoints no Fastify: `/consulta` (recebe intent e params, executa query).
- Lógica de permissão: checar perfil do usuário (hardcoded no JSON).

### ### Dia 3 – Integração IA (Extração de Intenção)

- Criar prompt de system para Haiku/Gemini:
  - > Você é um assistente que extrai intenção e parâmetros de perguntas sobre estoque e compras. Responda APENAS em JSON com os campos: intent, params (objeto). Intenções possíveis: 'estoque\_fornecedor', 'participacao\_fornecedor', 'comparacao\_periodo', 'margem', 'fora\_escopo'.
- Exemplo: ...
- Testar no Google AI Studio até ficar estável.
- Implementar chamada via SDK.

### ### Dia 4 – Formatação + Alertas + Gráficos

- Criar função de formatação: recebe dados da query e gera texto (pode ser via IA ou template).
- Gerar gráfico de barras/linhas com QuickChart (URL).
- Implementar alerta: se `quebra` (entrada - saldo) > 10% da entrada, exibe luz amarela.
- Adicionar contador de custo (estimar tokens e exibir no rodapé da resposta).

### ### Dia 5 – Web Chat + Fluxos de Recusa

- Construir UI com HTML/CSS/JS, servida via Fastify (rota `/`).
- Simular login: selecionar perfil (compras ou financeiro).
- Implementar recusa para:
  - Pergunta sem permissão → "Você não tem acesso a essa métrica."
  - Pergunta fora do catálogo → "Desculpe, não entendi. Posso ajudar com estoque, fornecedores ou margens."

- Falha simulada (ex: banco fora) → "Ocorreu um erro, tente novamente."

#### ### Dia 6 – Roteiro de Demo + Testes

- Preparar 5 perguntas-chave (incluindo a da Tramontina) que cubram todos os critérios.
- Testar cada critério de aceite (ver documento).
- Gravar um vídeo ou preparar script.

#### ### Dia 7 – Buffer/Polimento + Stretch (Google Chat)

- Ajustes finos de UI, mensagens de erro, performance.
- Se sobrar tempo, implementar integração com Google Chat (webhook) como extra.

---

### ## 6. Prompt de IA (System + User)

**\*\*System Prompt (para extração):\*\***

```

Você é um extrator de intenção para perguntas sobre dados de compras e estoque.

Identifique a intenção e extraia parâmetros.

Intenções:

- "estoque_fornecedor": pergunta sobre estoque atual e quebra de um fornecedor. Parâmetros: fornecedor (nome), periodo (mês/ano, opcional).
- "participacao_fornecedor": pergunta sobre participação de compras de um grupo fornecedor. Parâmetros: fornecedor_grupo (nome), periodo.
- "comparacao_periodo": pergunta comparando dois períodos para um produto/fornecedor. Parâmetros: item (produto ou fornecedor), periodo1, periodo2.
- "margem": pergunta sobre margem de lucro por produto/fornecedor. Parâmetros: item.
- "fora_escopo": quando não se encaixa.

Retorne APENAS JSON, ex: {"intent":"estoque_fornecedor","params":

{"fornecedor":"Tramontina","periodo":"2026-07"}}

```

**\*\*User Prompt:\*\*** a pergunta do usuário.

**\*\*Formatação (pode ser via IA ou template):\*\***

Se usar IA, prompt: "Formate os seguintes dados em uma resposta amigável para um usuário de compras: [dados]. Inclua um resumo, números e destaque a quebra se houver."

---

### ## 7. Dados Fictícios (Exemplo de 50 produtos)

Código	Nome	Categoria	Unidade
CAB10	Cabo 10mm <sup>2</sup>	Cabos	m
INT3P	Interruptor 3 pólos	Dispositivos	un
DIS16	Disjuntor 16A	Proteção	un
LED20	Lâmpada LED 20W	Iluminação	un
...	...	...	...

(Posso gerar a lista completa se precisar.)

---

### ## 8. Critérios de Aceite da Demo (todos verificáveis)

1. Pergunta "Tramontina, entradas e saldo deste mês" → mostra total entrado, saldo, quebra e alerta amarelo se >10%.
2. Estoque/giro de um fornecedor → resposta com número e gráfico.
3. Participação agrupando CNPJs → soma dos valores de todos os CNPJs do grupo.
4. Comparação de períodos → dois valores e variação %.
5. Usuário financeiro pergunta margem → OK. Usuário compras pergunta margem → recusado.
6. Pergunta fora do catálogo → "não entendi".
7. Toda resposta mostra custo estimado (ex: "Custo desta conversa: R\$0,02").
8. Resposta em  $\leq 8$  segundos.

---

### ## 9. Sugestões para a Apresentação

- Comece mostrando o chat com o perfil "compras".
- Faça a pergunta da Tramontina (clímax) → a quebra aparece e a luz amarela acende.
- Depois pergunte margem (recusado).
- Troque para perfil "financeiro" e repita a margem (liberado).
- Mostre o contador de custo.
- Finalize com "Isso tudo com dados sintéticos, mas a mesma arquitetura conecta ao seu ERP em 2 semanas."

---

### ## 10. Tecnologias Detalhadas

- **Backend:** Node 20 + Fastify.
- **Banco:** MySQL (Cloudez).
- **IA:** Gemini 2.5 Flash-Lite (mais barato) ou Haiku 4.5 (mais estável).
- **Gráficos:** QuickChart.io (URL).
- **Frontend:** HTML/CSS/JS puro (ou React se preferir).
- **Deploy:** Cloudez (sandbox).

---